

Fakeless

Project Team

Muhammad Tahir Sundhu	22I-0821
Nabeed Haider	22I-0871
Sameed Ahmed Siddiqui	22I-8223

Session 2022-2026

Supervised by

Dr. Syed Qaiser Ali Shah

Co-Supervised by

Mr. Almas Khan



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2025

Contents

1	Introduction	1
1.1	Problem Statement	2
1.2	Scope	2
1.3	Modules	3
1.3.1	Audio Processing and Perturbation Module	3
1.3.2	ML Optimization and Adaptation Module	4
1.3.3	Mobile Application Module (Client App)	4
1.3.4	Cloud Processing and API Module	4
1.3.5	Security and Privacy Module	5
1.4	User Classes and Characteristics	5
2	Project Requirements	7
2.1	Use-case/Event Response Table/Storyboarding	7
2.2	Use-case/Event Response Table/Storyboarding	7
2.3	Functional Requirements	12
2.3.1	Audio Processing and Perturbation Module	12
2.3.2	ML Optimization and Adaptation Module	12
2.3.3	Mobile Application Module	13
2.3.4	Cloud Processing and API Module	13
2.3.5	Security and Privacy Module	13
2.4	Non-Functional Requirements	13
2.4.1	Reliability	13
2.4.2	Usability	14
2.4.3	Performance	14
2.4.4	Security	14
2.5	Domain Model	14
3	System Overview	17
3.1	Architectural Design	17
3.2	Design Models	19
3.2.1	Activity Diagrams	20
3.2.2	Data Flow Diagram (Level 0–3)	28
3.2.3	System Sequence Diagrams	30

3.2.4	State Transition Diagram	39
3.3	Data Design	39
3.3.1	Data Storage Overview	39
3.3.2	Structures	40
3.3.3	Data Flow Description	40
3.3.4	Data Design Principles	40
References		42

List of Figures

2.1	Use Case Diagram for FakeLess Application	8
2.2	Domain Model for FakeLess Application	15
3.1	High-Level Architecture of FakeLess Application	18
3.2	Activity Diagram for Use Case 1: User Registration and Login	20
3.3	Activity Diagram for Use Case 2: Record Audio	21
3.4	Activity Diagram for Use Case 3: Upload Existing Audio	21
3.5	Activity Diagram for Use Case 4: Apply Voice Protection	22
3.6	Activity Diagram for Use Case 5: Adjust Protection Strength	22
3.7	Activity Diagram for Use Case 6: Playback and Compare Audio	23
3.8	Activity Diagram for Use Case 7: Export or Share Protected Audio	24
3.9	Activity Diagram for Use Case 8: Analyze Audio Quality Metrics	24
3.10	Activity Diagram for Use Case 9: Cloud Protection Mode	25
3.11	Activity Diagram for Use Case 10: Offline Local Protection	25
3.12	Activity Diagram for Use Case 11: Model Update Synchronization	26
3.13	Activity Diagram for Use Case 12: Manage User Preferences	26
3.14	Activity Diagram for Use Case 13: Error Detection and Logging	27
3.15	Activity Diagram for Use Case 14: Privacy Mode Activation	27
3.16	Activity Diagram for Use Case 15: Logout and Session Termination	28
3.17	Level 0 Data Flow Diagram — Context Diagram of the FakeLess System	28
3.18	Level 1 DFD — Major Processes of FakeLess System	29
3.19	Level 2 DFD — Quality Evaluation Workflow	29
3.20	Level 2 DFD — Audio Protection Workflow	30
3.21	Level 3 DFD — ML Perturbation Process	30
3.22	SSD for Use Case 1: User Registration and Login	31
3.23	SSD for Use Case 2: Record Audio	31
3.24	SSD for Use Case 3: Upload Existing Audio	32
3.25	SSD for Use Case 4: Apply Voice Protection	33
3.26	SSD for Use Case 5: Adjust Protection Strength	33
3.27	SSD for Use Case 6: Playback and Compare Audio	34
3.28	SSD for Use Case 7: Export or Share Protected Audio	34
3.29	SSD for Use Case 8: Analyze Audio Quality Metrics	35
3.30	SSD for Use Case 9: Cloud Protection Mode	35

3.31 SSD for Use Case 10: Offline Local Protection	36
3.32 SSD for Use Case 11: Model Update Synchronization	36
3.33 SSD for Use Case 12: Manage User Preferences	37
3.34 SSD for Use Case 13: Error Detection and Logging	37
3.35 SSD for Use Case 14: Privacy Mode Activation	38
3.36 SSD for Use Case 15: Logout and Session Termination	38
3.37 State Transition Diagram for Audio Processing States	39

Chapter 1

Introduction

FakeLess is a mobile-first application designed to prevent unauthorized voice cloning and audio deepfake misuse. In the era of generative AI, cloning a person's voice requires only a few seconds of audio, creating significant risks such as fraud, impersonation, and misinformation. FakeLess applies imperceptible, machine-learning-driven perturbations to recorded or uploaded audio files, making them unusable by deepfake models while preserving sound quality. The application combines advanced machine learning (ML) methods such as Reinforcement Learning (RL) and Generative Adversarial Networks (GANs) to generate protective noise patterns that are undetectable to human listeners but effective against deepfake architectures like Tacotron 2, WaveNet, SV2TTS, and RVC.

This document specifies the system requirements and architecture of FakeLess for various stakeholders. It is intended for:

- **Developers:** Responsible for designing, coding, and integrating the FakeLess mobile and backend components. This document helps developers understand the system's architecture, APIs, and module interactions, as well as the machine learning algorithms used for generating adversarial perturbations. It ensures that implementation follows the defined data flow, module boundaries, and security standards.
- **Supervisors:** Oversee the project's timeline, deliverables, and resource allocation. For them, this document outlines milestones, dependencies, and system modules, enabling effective tracking of development progress and ensuring that all components align with the project's overall objectives and technical feasibility.
- **Testers:** Tasked with validating the performance, accuracy, and robustness of FakeLess under different conditions. This report provides them with detailed insights into input/output specifications, expected behaviors, and error handling mechanisms, helping them design systematic test cases and evaluate the system's compliance with quality benchmarks such as PESQ, STOI, and MOS.

- **End Users:** Include journalists, content creators, business professionals, educators, and general individuals seeking to protect their voice identity from deepfake misuse. For this group, FakeLess offers a simple yet effective solution to apply voice protection without compromising natural audio quality. The document helps users understand the purpose, workflow, and privacy benefits of the system.
- **Researchers:** Represent the scientific community interested in adversarial learning and digital security. The document provides them with a practical implementation reference for adversarial perturbation in the audio domain, offering valuable insights into defensive ML strategies, hybrid client-server architecture, and the impact of perturbation strength on voice intelligibility.

1.1 Problem Statement

The increasing use of deepfake technologies has made it possible to generate highly realistic voice clones from minimal audio samples. This poses threats to personal privacy, security, and trust in digital communications. Many individuals and organizations have already faced fraud, identity theft, and misinformation due to cloned voices. [1, 2]. Current solutions mainly focus on detection—identifying fake content after creation—rather than preventing it. These detection systems are reactive, enterprise-oriented, and not accessible to the general public. [6, 7, 8, 11]

FakeLess aims to solve this by introducing a proactive, consumer-focused solution that prevents deepfakes from being generated in the first place. By applying imperceptible, ML-optimized perturbations to user audio, FakeLess ensures that even if an attacker obtains a recording, the cloned output will fail or produce unusable results. The app combines local processing for privacy with cloud-based ML ensembles for high computational tasks. This hybrid design gives users immediate protection while maintaining audio clarity. The ultimate goal is to empower everyday users—content creators, professionals, and privacy-conscious individuals—to protect their digital voice identity against unauthorized cloning.

1.2 Scope

The scope of FakeLess encompasses the research, design, and development of a mobile-based system that prevents unauthorized voice cloning through adaptive, machine-learning-driven perturbations. Users can record or upload audio, select protection strength, and export the processed version. The system operates in both local and cloud modes: lightweight local protection for short clips and cloud processing for heavy, high-quality protection.

The backend uses PyTorch-based models deployed via FastAPI, supporting dynamic reinforcement learning and GAN frameworks.

Core functionalities include:

- Generation of imperceptible perturbations in time and frequency domains.
- Optimization of perturbations through RL and GAN models.
- Maintenance of natural sound quality using psychoacoustic masking.
- Real-time audio visualization and protection-level controls in the mobile app.
- Robust quality assessment using metrics like PESQ, STOI, and MOS.
- Hybrid local-cloud architecture ensuring privacy and scalability.

Out of scope are detection-only systems, passive watermarking, or generic deepfake detectors. FakeLess focuses exclusively on proactive prevention—ensuring that users’ voices remain secure and cannot be cloned by any known or emerging deepfake models.

1.3 Modules

The FakeLess system is modular and comprises several interconnected components that collectively deliver real-time, robust, and privacy-preserving voice protection.

1.3.1 Audio Processing and Perturbation Module

This is the core module responsible for analyzing input audio, extracting features, and generating adversarial perturbations.

1. Real-time audio recording and preprocessing (denoising, normalization, segmentation).
2. Feature extraction (MFCCs, spectral centroid, bandwidth, zero-crossing rate).
3. Time- and frequency-domain perturbation application.
4. Psychoacoustic masking to maintain intelligibility and sound quality.
5. Quality verification through PESQ, STOI, and MOS metrics.

1.3.2 ML Optimization and Adaptation Module

This module optimizes protection dynamically using reinforcement and adversarial learning.

1. Reinforcement Learning (PPO/SAC) engine for adaptive perturbation tuning.
2. GAN-based perturbation generation to counter adaptive deepfake models.
3. Neural Network Search for discovering optimal perturbation operations.
4. Ensemble integration combining RL and GAN results for robustness.
5. Continuous model updates to stay resilient against evolving attacks.

1.3.3 Mobile Application Module (Client App)

The user-facing mobile interface that enables interaction with the FakeLess system.

1. Record or upload audio clips for protection.
2. Adjustable protection strength slider and live audio visualization.
3. Before/after comparison playback for quality verification.
4. Cross-platform (Android/iOS) compatibility via React Native.

1.3.4 Cloud Processing and API Module

Handles advanced ML computation and enterprise-level batch processing.

1. PyTorch-based backend hosted on AWS/GCP.
2. FastAPI endpoints for secure audio upload and retrieval.
3. Scalable, asynchronous task handling using Celery and Redis.
4. Data encryption and secure model updates.

User Class	Description
General User	Any individual (content creator, professional, or student) who wishes to protect their voice recordings from cloning. Requires minimal technical knowledge.
Enterprise User	Corporate or institutional client (e.g., media, legal, or government) needing large-scale or automated protection via APIs. Focused on batch processing and compliance.
Researcher/Developer	AI/ML cybersecurity professionals using the API for experimental or research purposes. Requires understanding of deepfake models and adversarial ML.
Tester/QA Engineer	Ensures app reliability, protection efficiency, and quality standards through repeated testing of ML models and audio outputs.

1.3.5 Security and Privacy Module

Ensures protection of user data throughout all processes.

1. Local-first processing to minimize data transfer.
2. Encrypted communication and anonymized analytics.
3. Compliance with privacy standards such as GDPR.

1.4 User Classes and Characteristics

FakeLess targets both individual and organizational users, ensuring accessibility and privacy across different use cases.

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-case/Event Response Table/Storyboarding

As FakeLess is an interactive end-user mobile application, the most suitable requirement representation is through **Use Cases**. Each use case defines how a user interacts with the system to achieve a specific goal, ensuring clarity for developers and testers.

2.2 Use-case/Event Response Table/Storyboarding

FakeLess is an interactive end-user mobile application. Therefore, the **Use Case** approach is best suited to model its functional behavior. Each use case describes a distinct user interaction or system process that supports FakeLess's primary goal: preventing audio deepfake misuse through voice protection.

Use Case Diagram

Detailed Use Cases

Use Case 1: User Registration and Login

- **Actor:** User
- **Description:** Allows the user to create an account and log in using email, password, or Google sign-in.

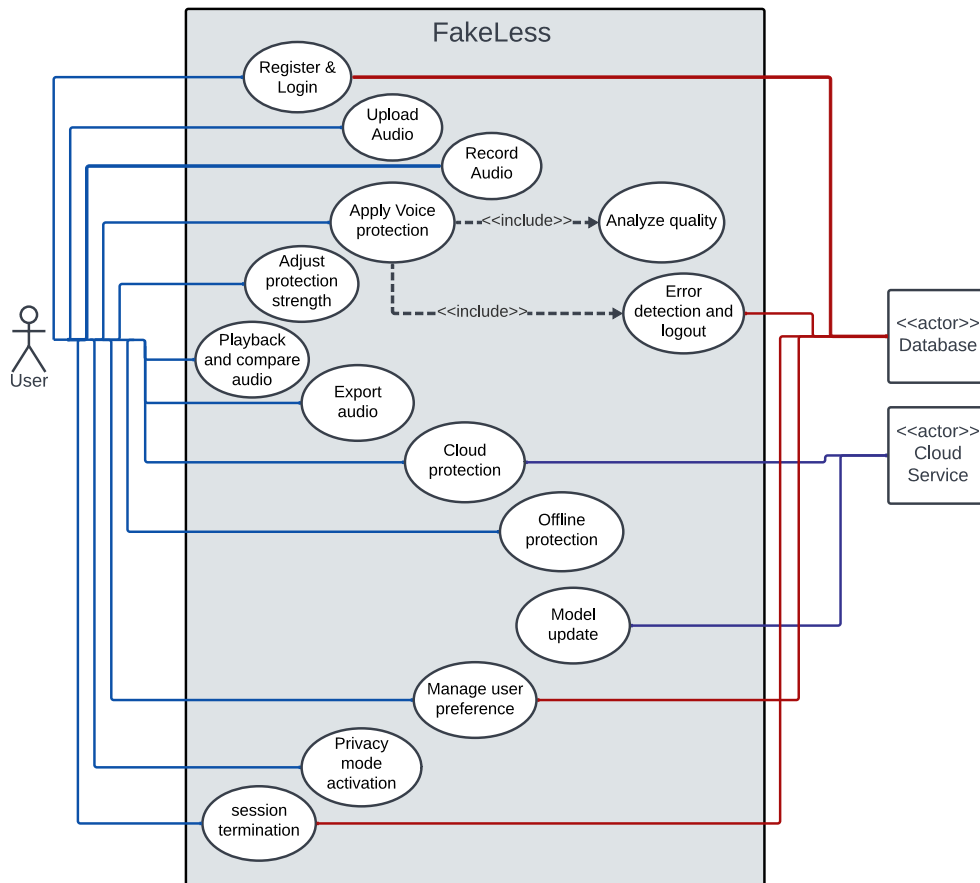


Figure 2.1: Use Case Diagram for FakeLess Application

- **Precondition:** Internet connectivity available.
- **Postcondition:** User profile created or authenticated successfully.
- **Main Flow:**
 1. User launches the FakeLess app.
 2. Selects “Sign Up” or “Login”.
 3. Enters credentials or authenticates via Google.
 4. System validates information and grants access.

Use Case 2: Record Audio

- **Actor:** User
- **Description:** User records a voice sample for protection.

- **Precondition:** Microphone permission granted.
- **Postcondition:** Audio file stored locally and ready for processing.

Use Case 3: Upload Existing Audio

- **Actor:** User
- **Description:** User uploads an existing voice file (e.g., podcast, message, or speech).
- **Precondition:** File must be in a supported format (WAV, MP3, AAC).
- **Postcondition:** Audio stored securely and displayed in the library.

Use Case 4: Apply Voice Protection

- **Actor:** User
- **Description:** User applies ML-generated perturbations to prevent voice cloning.
- **Precondition:** Audio file selected and valid.
- **Postcondition:** Protected audio generated and stored.

Use Case 5: Adjust Protection Strength

- **Actor:** User
- **Description:** User selects desired protection intensity (Low, Medium, High).
- **Precondition:** Audio ready for processing.
- **Postcondition:** System updates perturbation parameters accordingly.

Use Case 6: Playback and Compare Audio

- **Actor:** User
- **Description:** User compares the original and protected versions to assess audio quality.
- **Precondition:** Both versions available in the app.
- **Postcondition:** User confirms or reprocesses if unsatisfied.

Use Case 7: Export or Share Protected Audio

- **Actor:** User
- **Description:** Allows export of protected audio to local storage or sharing via other apps.
- **Precondition:** Protection process completed successfully.
- **Postcondition:** File exported securely in chosen format.

Use Case 8: Analyze Audio Quality Metrics

- **Actor:** System
- **Description:** The system calculates metrics like PESQ, STOI, and MOS to verify intelligibility.
- **Precondition:** Protected audio generated.
- **Postcondition:** Quality report displayed to the user.

Use Case 9: Cloud Protection Mode

- **Actor:** User / Cloud Service
- **Description:** User opts for cloud-based protection for large audio files.
- **Precondition:** Internet connectivity and authenticated user.
- **Postcondition:** Cloud ML model applies protection and returns result.

Use Case 10: Offline Local Protection

- **Actor:** User
- **Description:** User performs lightweight local protection without internet access.
- **Precondition:** Audio file under 2 MB.
- **Postcondition:** Locally protected file generated.

Use Case 11: Model Update Synchronization

- **Actor:** Cloud Backend
- **Description:** The app synchronizes its ML models with the cloud to stay resilient against new deepfake architectures.

- **Precondition:** Network available and update scheduled.
- **Postcondition:** Models updated to latest version.

Use Case 12: Manage User Preferences

- **Actor:** User
- **Description:** User configures settings such as default protection level, language, and dark/light theme.
- **Precondition:** Logged-in session active.
- **Postcondition:** Preferences saved locally and encrypted.

Use Case 13: Error Detection and Logging

- **Actor:** System internal triggered
- **Description:** Logs errors during audio processing for debugging and reliability.
- **Precondition:** Error or failure event triggered.
- **Postcondition:** Log entry stored securely for diagnostics.

Use Case 14: Privacy Mode Activation

- **Actor:** User
- **Description:** Enables privacy mode to restrict analytics collection and disable cloud sync.
- **Precondition:** Logged-in user selects “Privacy Mode.”
- **Postcondition:** System switches to fully local operation.

Use Case 15: Logout and Session Termination

- **Actor:** User
- **Description:** Ends the user session securely and clears temporary audio data.
- **Precondition:** User logged in.
- **Postcondition:** Session terminated; cached files deleted.

2.3 Functional Requirements

The following are the core functional requirements of FakeLess. These requirements are categorized according to the primary system modules defined in Chapter 1.

2.3.1 Audio Processing and Perturbation Module

Following are the requirements for this module:

1. **FR1:** The system shall record or accept uploaded audio files in supported formats (WAV, MP3, AAC, FLAC).
2. **FR2:** The system shall perform preprocessing including denoising, normalization, and segmentation.
3. **FR3:** The system shall extract features such as MFCCs, spectral centroid, and zero-crossing rate.
4. **FR4:** The system shall generate perturbations in time and frequency domains using ML algorithms.
5. **FR5:** The system shall ensure audio quality preservation using psychoacoustic masking.

2.3.2 ML Optimization and Adaptation Module

Following are the requirements for this module:

1. **FR1:** The system shall implement a Reinforcement Learning agent (PPO/SAC) to optimize perturbation parameters.
2. **FR2:** The system shall employ a GAN model to create adversarial perturbations.
3. **FR3:** The system shall perform ensemble integration combining RL and GAN outputs.
4. **FR4:** The system shall continuously update ML models to adapt to new deepfake architectures.

2.3.3 Mobile Application Module

Following are the requirements for this module:

1. **FR1:** The app shall provide a user-friendly interface with real-time audio visualization.
2. **FR2:** The app shall allow users to control protection strength through a slider.
3. **FR3:** The app shall display before/after comparisons for user validation.
4. **FR4:** The app shall notify users of processing status and completion.

2.3.4 Cloud Processing and API Module

Following are the requirements for this module:

1. **FR1:** The backend shall provide FastAPI endpoints for audio upload, processing, and retrieval.
2. **FR2:** The system shall use PyTorch for ML computation and Redis for caching.
3. **FR3:** The system shall handle large audio files through batch processing.
4. **FR4:** The system shall encrypt all transmitted data between client and server.

2.3.5 Security and Privacy Module

Following are the requirements for this module:

1. **FR1:** The system shall process user audio locally by default.
2. **FR2:** The system shall ensure compliance with privacy regulations (GDPR).
3. **FR3:** The system shall anonymize analytics and user activity data.
4. **FR4:** The system shall securely store and encrypt user preferences.

2.4 Non-Functional Requirements

2.4.1 Reliability

- **NFR1:** The system shall have a mean time between failures (MTBF) of at least 500 hours of continuous operation.

- **NFR2:** The app shall recover automatically from processing interruptions by saving session data.
- **NFR3:** All processing errors shall be logged and reported for diagnostic review.

2.4.2 Usability

- **NFR1:** The user shall be able to record, protect, and export audio in fewer than five interactions.
- **NFR2:** The mobile UI shall conform to accessibility guidelines (WCAG 2.1).
- **NFR3:** First-time users shall understand the core workflow without formal training.

2.4.3 Performance

- **NFR1:** The system shall process 1 minute of audio in less than 5 seconds under standard conditions.
- **NFR2:** The app shall maintain less than 200 MB RAM usage during local processing.
- **NFR3:** Cloud processing shall handle up to 100 simultaneous users with auto-scaling.

2.4.4 Security

- **NFR1:** All user data and communications shall be encrypted using TLS 1.3.
- **NFR2:** The system shall not store unprotected or raw audio on any remote server.
- **NFR3:** The system shall implement token-based authentication for API access.
- **NFR4:** The app shall employ permission-based access control for recording and storage.

2.5 Domain Model

The domain model represents the key entities and their relationships in the FakeLess system.

Entities:

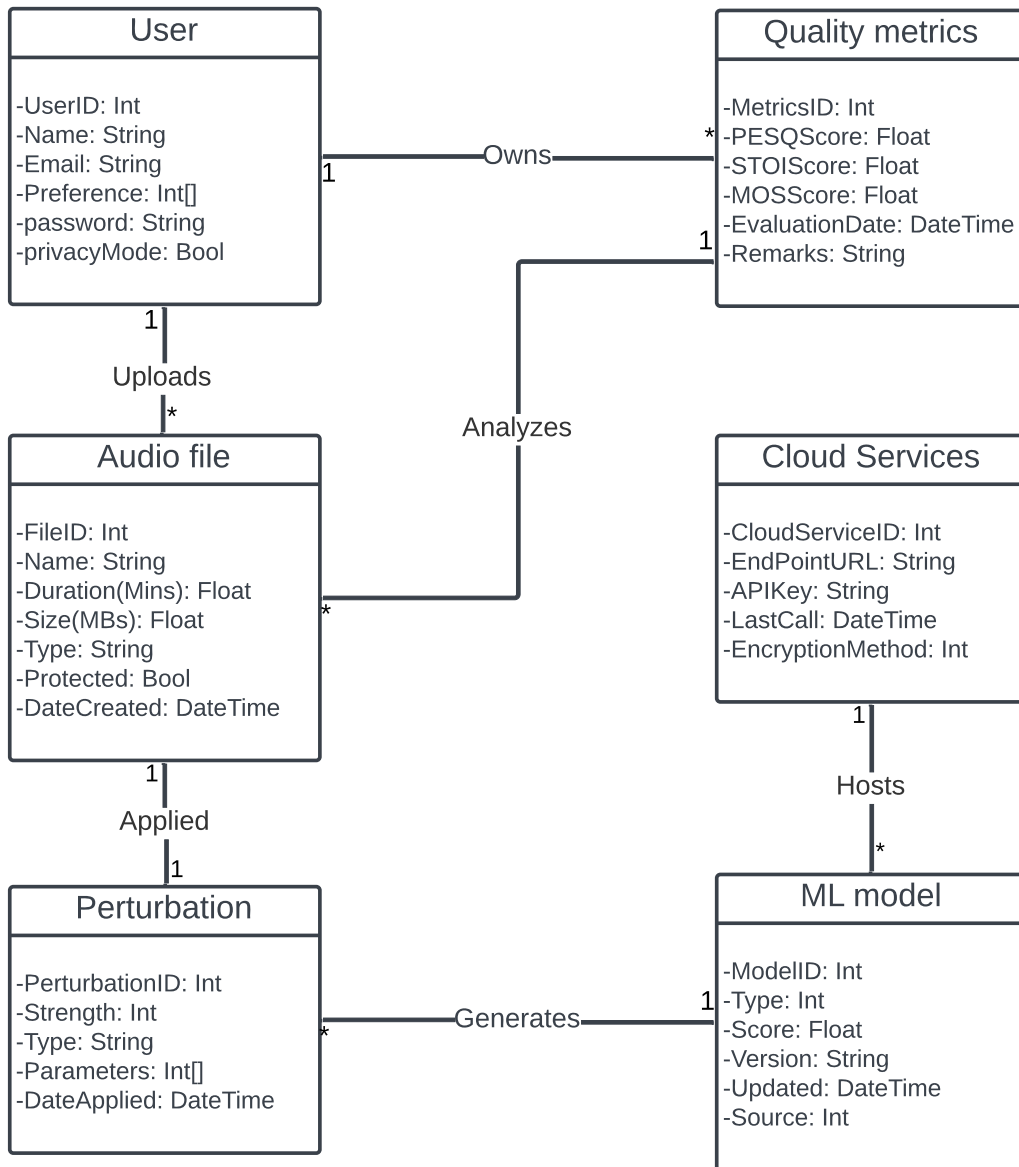


Figure 2.2: Domain Model for FakeLess Application

- **User:** Represents an app user with account and preferences.
- **AudioFile:** Stores metadata of uploaded or recorded files.
- **Perturbation:** Contains details of applied perturbations and parameters.
- **MLModel:** Represents RL and GAN instances used for protection.
- **QualityMetrics:** Holds values like PESQ, STOI, and MOS.

- **CloudService:** Represents backend APIs managing audio uploads and responses.

Chapter 3

System Overview

FakeLess is a mobile-based voice protection system that prevents the misuse of human speech through audio deepfake cloning. The application integrates machine learning models that generate subtle, imperceptible perturbations in a user's voice to make it unusable by voice-cloning algorithms while maintaining the natural sound quality. This proactive approach contrasts with conventional deepfake detection systems that act after a fake is generated.

The system employs a hybrid architecture consisting of a mobile client (built in React Native) and a backend (developed using Python FastAPI and PyTorch). Locally, the mobile app handles recording, playback, preprocessing, and lightweight protection, while the backend performs high-intensity ML computations such as adversarial training and reinforcement-based perturbation tuning. This modular design enables efficient processing, scalability, and user data privacy.

FakeLess helps users such as journalists, content creators, business professionals, and everyday individuals to secure their voice identity. It ensures that even if their audio is intercepted, it cannot be cloned or synthetically reproduced by models like Tacotron 2[9], WaveNet[10], or RVC[4]

3.1 Architectural Design

The architecture of FakeLess follows a **Client-Server (Hybrid Multi-tiered)** pattern , where the mobile application (client) interacts with a cloud backend server for machine learning operations. The system is decomposed into independent yet collaborative modules that perform audio preprocessing, perturbation generation, model optimization, and secure data handling.

At a high level, FakeLess consists of the following components:

- **Mobile Client Layer:** Handles recording, uploading, playback, and visualization of protected audio.
- **Application Layer:** Manages protection logic, ML inference calls, and data synchronization.
- **ML Model Layer:** Contains reinforcement learning and GAN-based models that generate protective perturbations.
- **Cloud Backend:** Implements FastAPI services for secure ML model hosting and asynchronous task execution.
- **Database Layer:** Stores metadata, user profiles, and model configurations.

High-Level Architectural Overview

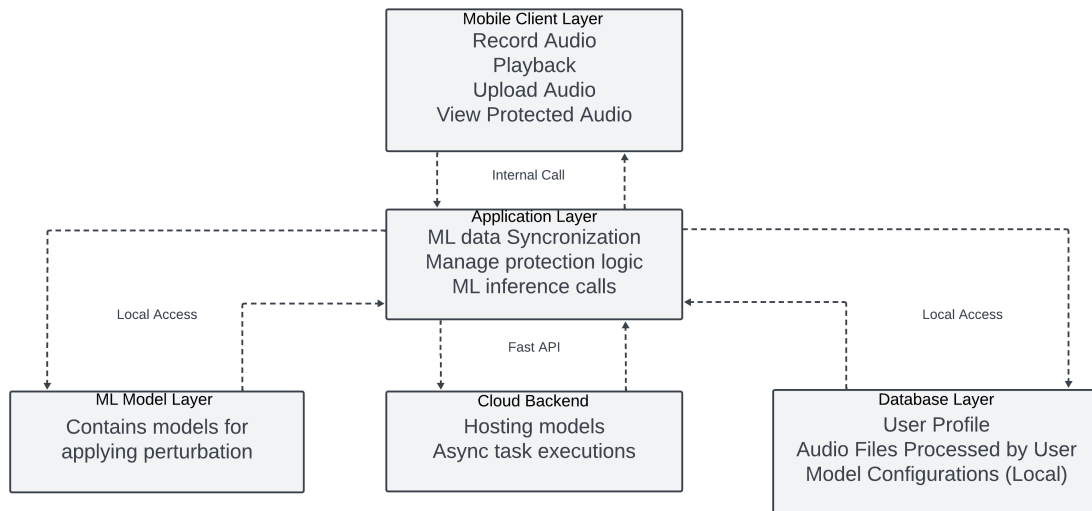


Figure 3.1: High-Level Architecture of FakeLess Application

System Decomposition

FakeLess is divided into independent yet cooperative procedural modules, each performing a specific function within the workflow. These modules are interconnected through well-defined function calls, forming a clear data flow pipeline.

System Modules

The FakeLess application is divided into several modular components, each responsible for a specific part of the procedural workflow. This modular decomposition ensures maintainability, scalability, and ease of debugging across both mobile and backend components [5, 13].

- **Audio Processing Module:** Handles recording, uploading, and preprocessing through functions like `record_audio()`, `upload_file()`, and `preprocess_audio()`. This module interacts with local storage to cache recordings before forwarding them for protection [3].
- **Protection Module:** Applies ML-generated perturbations using functions like `apply_protection()` and `adjust_strength()`. The perturbation logic draws from adversarial audio research and modern preventive frameworks such as AntiFake and SafeEar [12, 13].
- **Quality Evaluation Module:** Computes intelligibility and quality metrics (PESQ, STOI, MOS) using `calculate_metrics()` and `compare_audio()`. These standardized metrics ensure intelligibility while preserving natural speech quality [? ?].
- **Export Module:** Saves or shares protected audio through `export_audio()` and `share_file()`. The export process ensures encrypted file handling and controlled access for end users.
- **Cloud Communication Module:** Connects to backend services through API calls like `send_to_cloud()` and `receive_result()`. It manages heavy ML computations through asynchronous REST APIs developed with FastAPI and PyTorch backends [? ?].
- **Error Handling Module:** Captures and logs issues using `log_error()` and `display_error()`. The system maintains persistent logs for debugging and cloud synchronization of anonymized diagnostic data.

Each module works independently but passes its output as input to the next function, following a linear procedural flow.

3.2 Design Models

FakeLess follows a Procedural Design Approach, where the focus is on processes, control flow, and data movement rather than objects and inheritance. Each component of the system is implemented as a set of functions grouped under logical modules.

3.2.1 Activity Diagrams

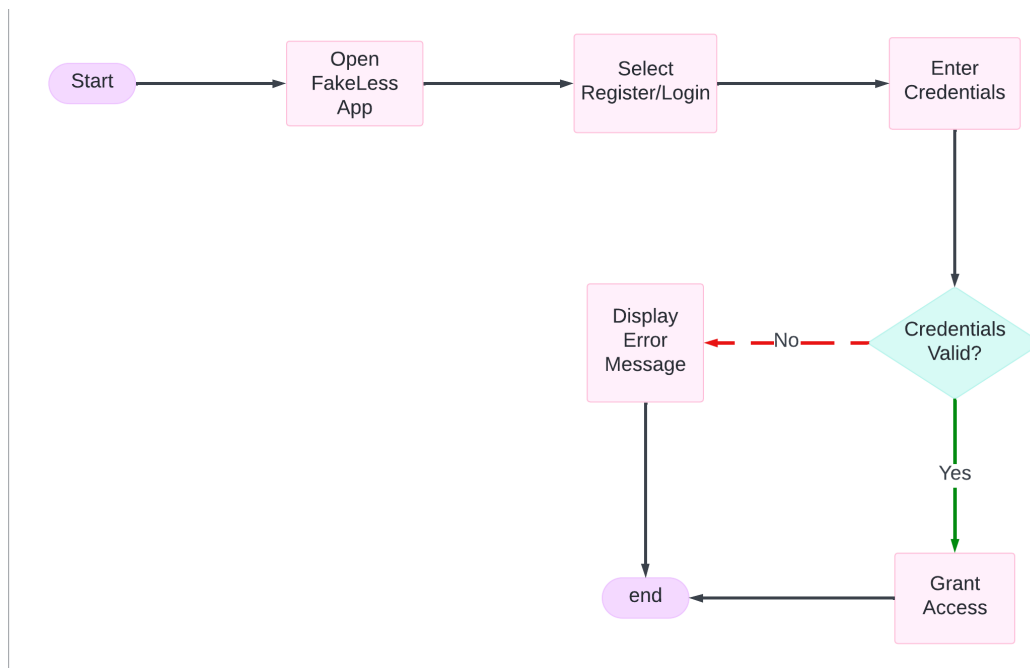


Figure 3.2: Activity Diagram for Use Case 1: User Registration and Login

Description: The process begins when the user opens the FakeLess app and selects either “Register” or “Login.” The system verifies the entered credentials with the backend and, if valid, grants access to the main interface; otherwise, an error message is shown.

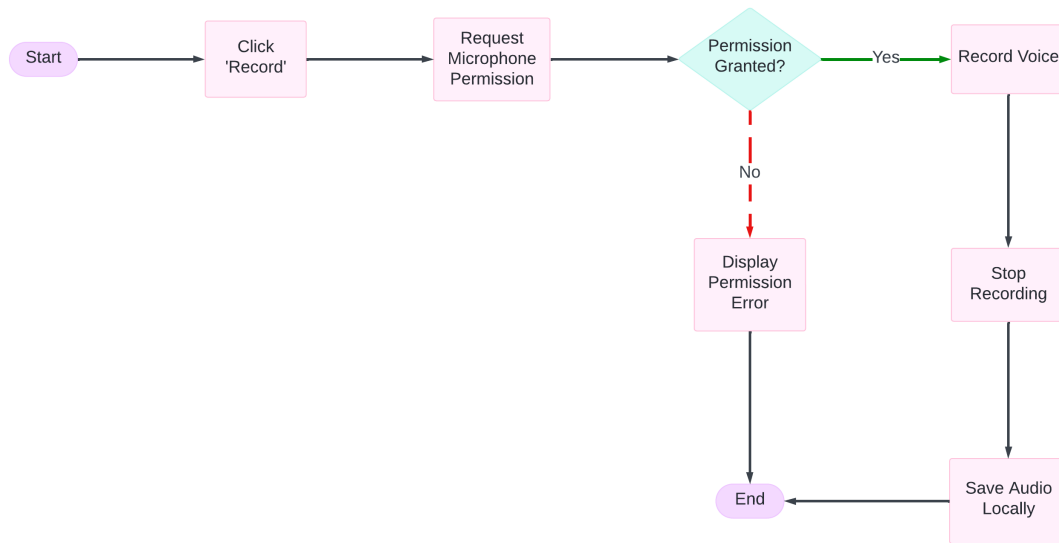


Figure 3.3: Activity Diagram for Use Case 2: Record Audio

Description: The user starts recording through the app, which first requests microphone permission. Once granted, the system records the voice input, stops upon user action, and saves the file locally for further use.

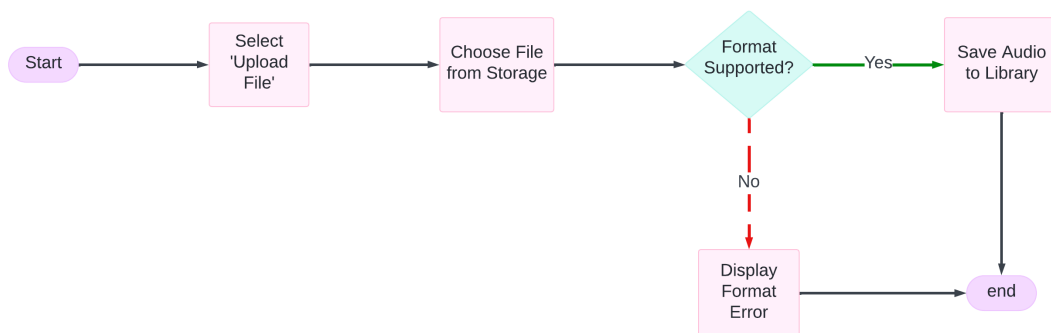


Figure 3.4: Activity Diagram for Use Case 3: Upload Existing Audio

Description: The user selects an existing voice file from the device. The system validates the format (WAV, MP3, AAC) and, if supported, stores the file in the library; otherwise, it prompts an error.

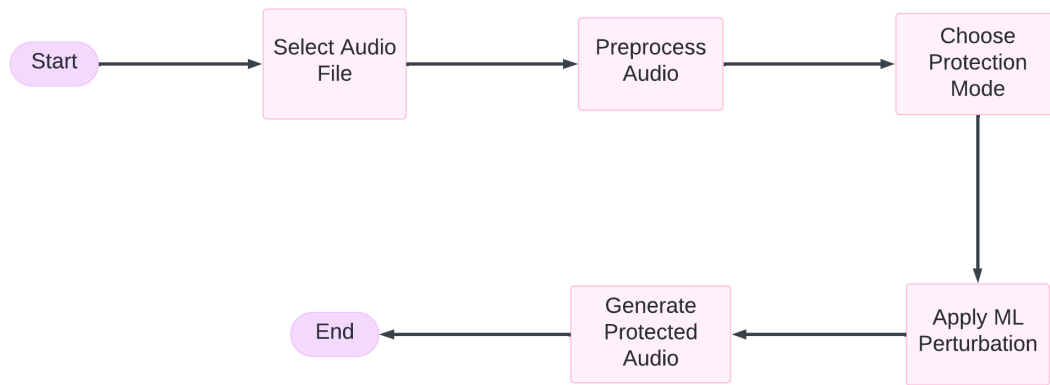


Figure 3.5: Activity Diagram for Use Case 4: Apply Voice Protection

Description: After selecting an audio file, the system preprocesses it and applies ML-generated perturbations either locally or through the cloud backend, generating a protected audio output.

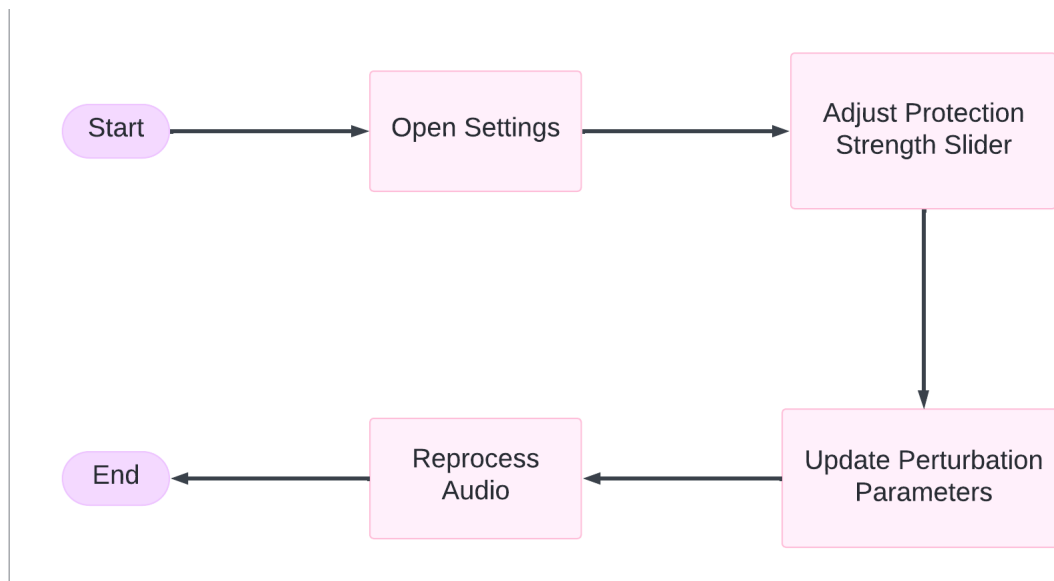


Figure 3.6: Activity Diagram for Use Case 5: Adjust Protection Strength

Description: The user modifies the protection intensity using a slider (Low, Medium, High). The system adjusts the perturbation parameters accordingly and reprocesses the audio.

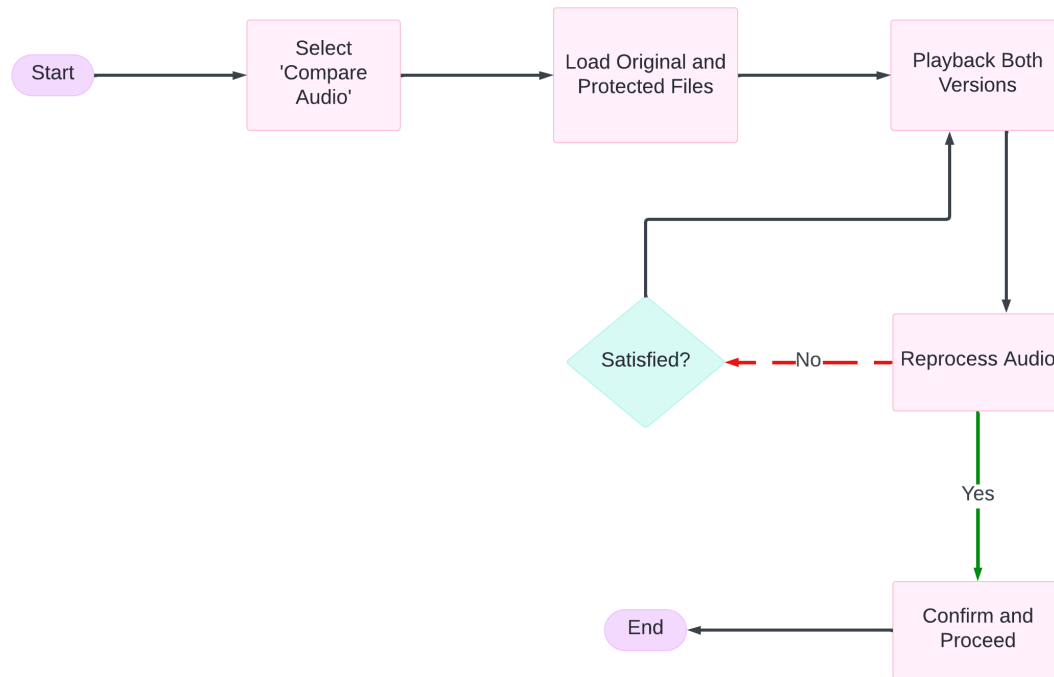


Figure 3.7: Activity Diagram for Use Case 6: Playback and Compare Audio

Description: The app retrieves both original and protected versions of the audio. The user plays and compares them, confirming satisfaction or choosing to regenerate protection if needed.

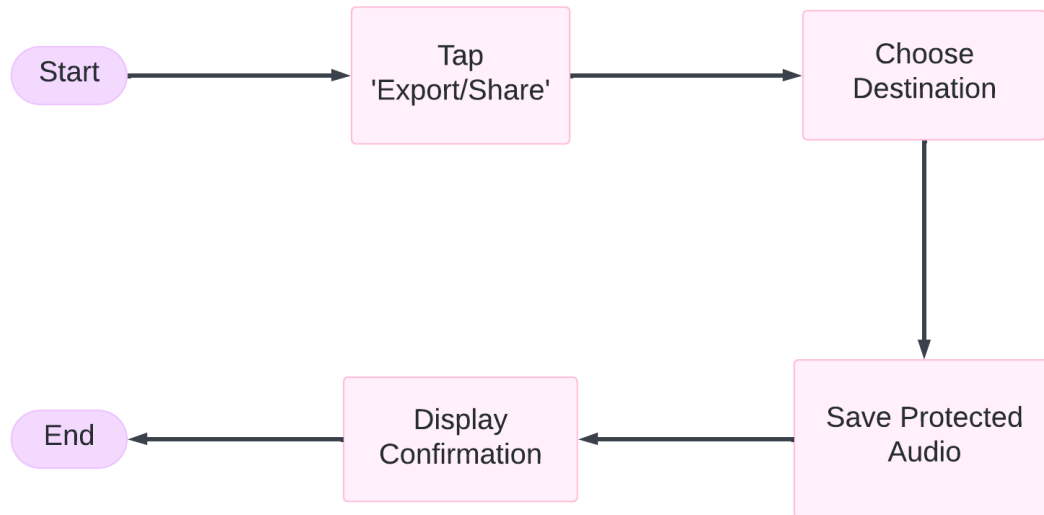


Figure 3.8: Activity Diagram for Use Case 7: Export or Share Protected Audio

Description: Once the file is protected, the user selects export or share options. The app securely saves the file in local storage or shares it through supported external apps.

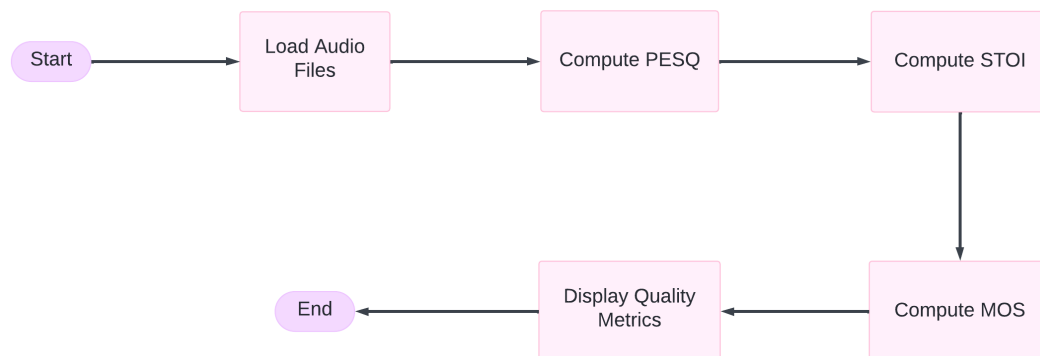


Figure 3.9: Activity Diagram for Use Case 8: Analyze Audio Quality Metrics

Description: The system loads both original and protected files and calculates quality metrics such as PESQ, STOI, and MOS. The results are displayed as a detailed report for the user.

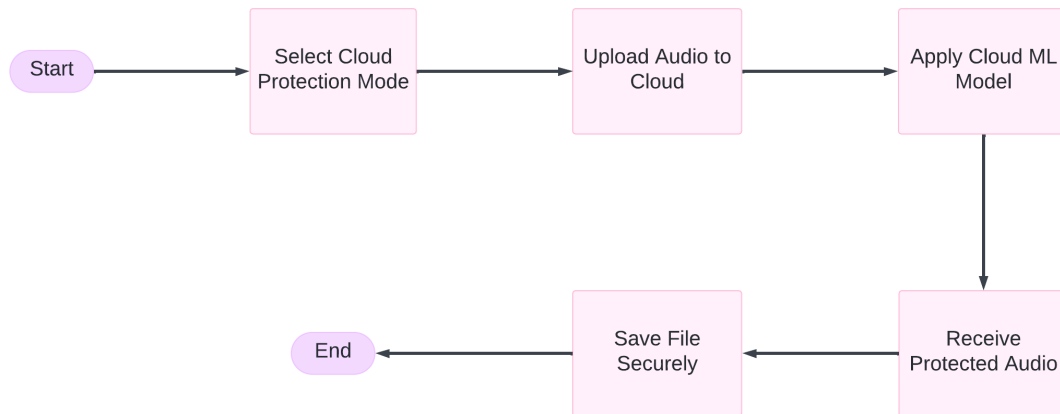


Figure 3.10: Activity Diagram for Use Case 9: Cloud Protection Mode

Description: In cloud mode, the audio file is uploaded to the backend. The ML engine applies high-intensity perturbations, and the protected file is returned and stored securely.

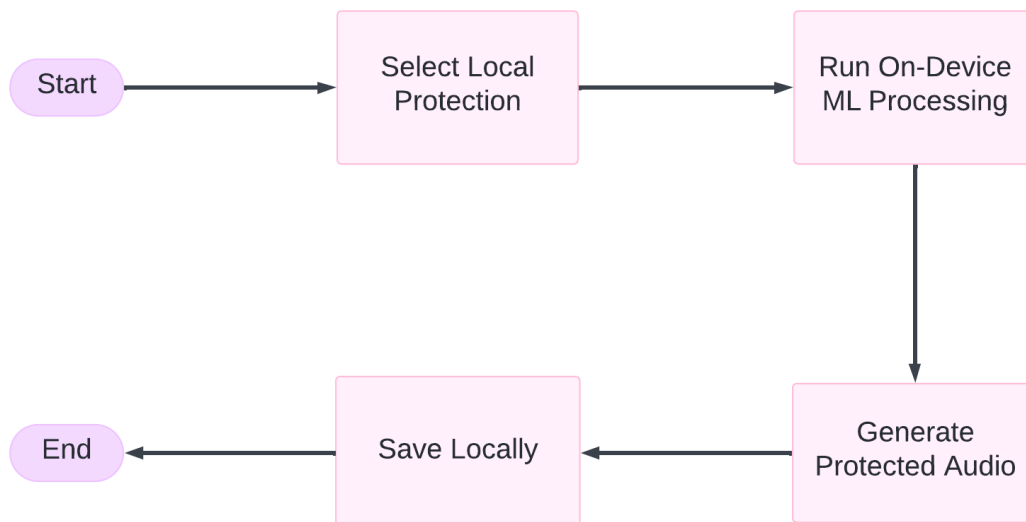


Figure 3.11: Activity Diagram for Use Case 10: Offline Local Protection

Description: If the file is small, the app performs local protection using embedded ML functions without an internet connection, ensuring quick processing and privacy.

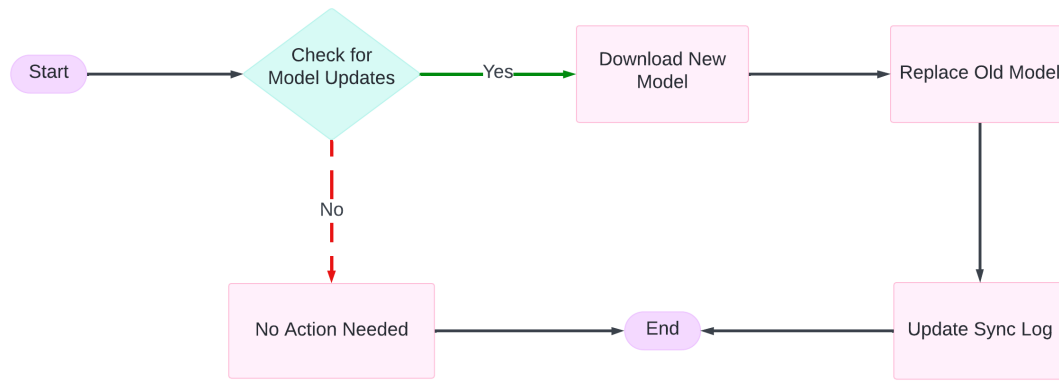


Figure 3.12: Activity Diagram for Use Case 11: Model Update Synchronization

Description: The app periodically checks for updated ML models on the cloud. If an update is available, it downloads and replaces the old version while logging the synchronization details.

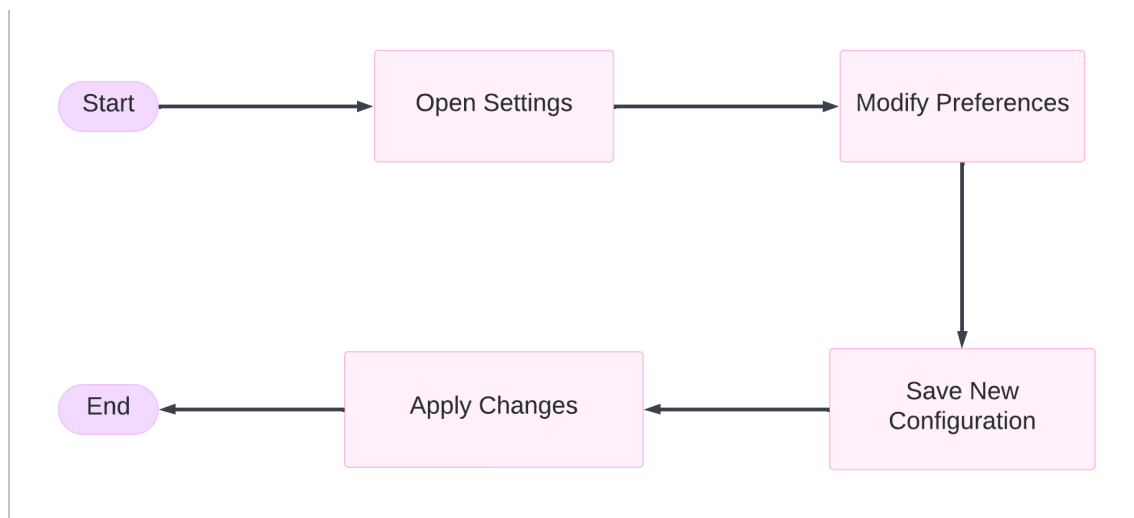


Figure 3.13: Activity Diagram for Use Case 12: Manage User Preferences

Description: The user opens the settings menu to modify preferences such as theme, language, and default protection strength. The app saves and applies changes immediately.

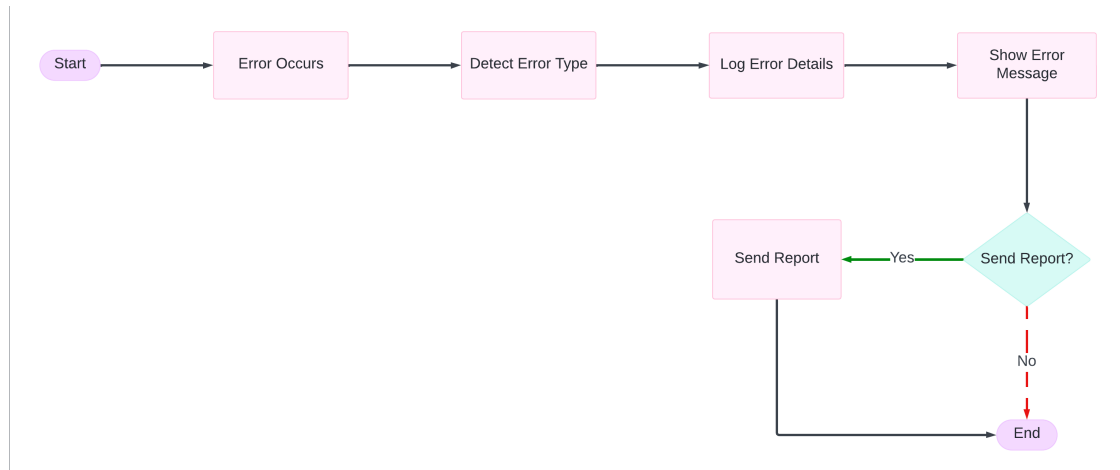


Figure 3.14: Activity Diagram for Use Case 13: Error Detection and Logging

Description: When an error occurs, the app detects it, records details in a log, and displays a notification. Optionally, anonymized reports can be sent to the backend for analysis.

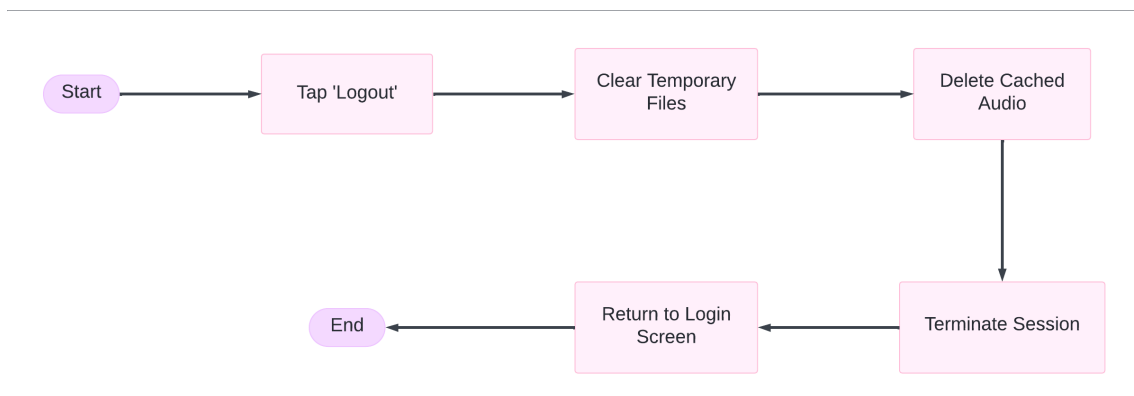


Figure 3.15: Activity Diagram for Use Case 14: Privacy Mode Activation

Description: When the user activates Privacy Mode, analytics collection and cloud synchronization are disabled. The app switches all operations to local-only mode for enhanced privacy.

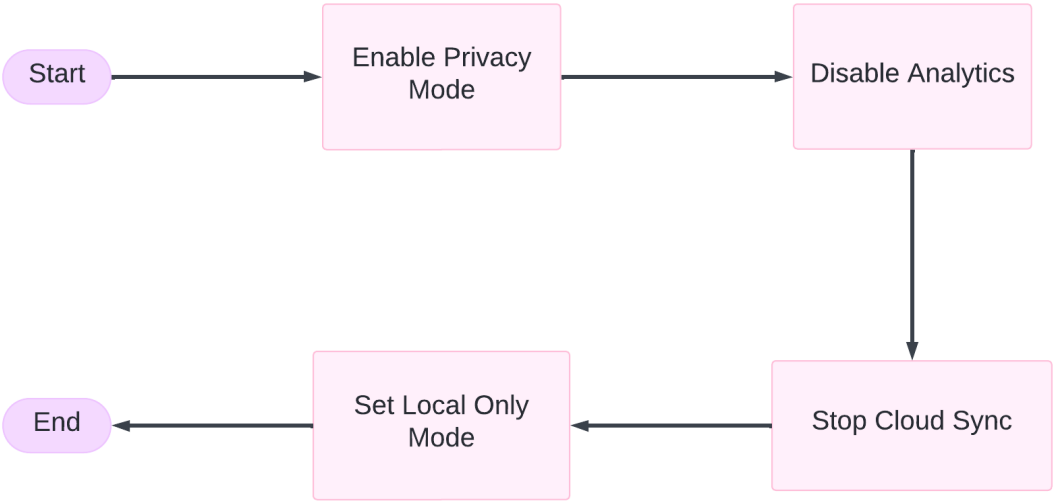


Figure 3.16: Activity Diagram for Use Case 15: Logout and Session Termination

Description: The user selects “Logout.” The system clears temporary audio data, deletes cached files, terminates the session, and returns to the login screen securely.

3.2.2 Data Flow Diagram (Level 0–3)

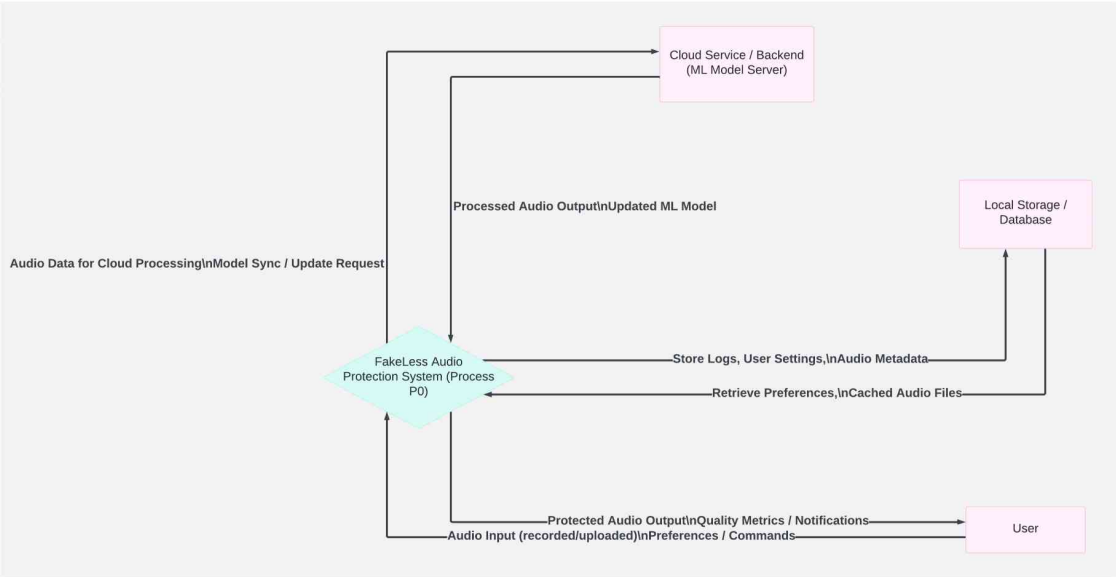


Figure 3.17: Level 0 Data Flow Diagram — Context Diagram of the FakeLess System

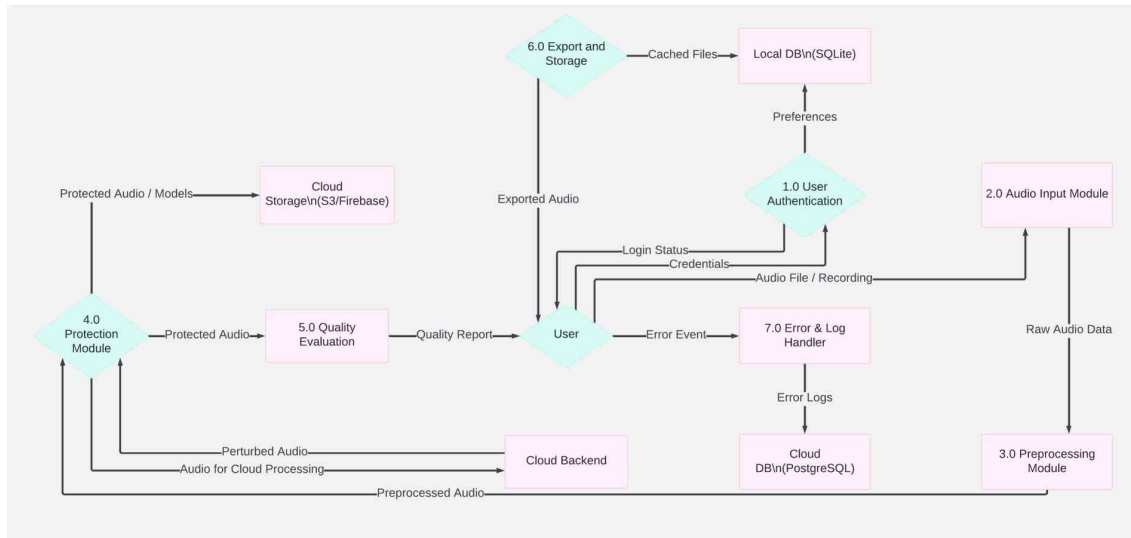


Figure 3.18: Level 1 DFD — Major Processes of FakeLess System

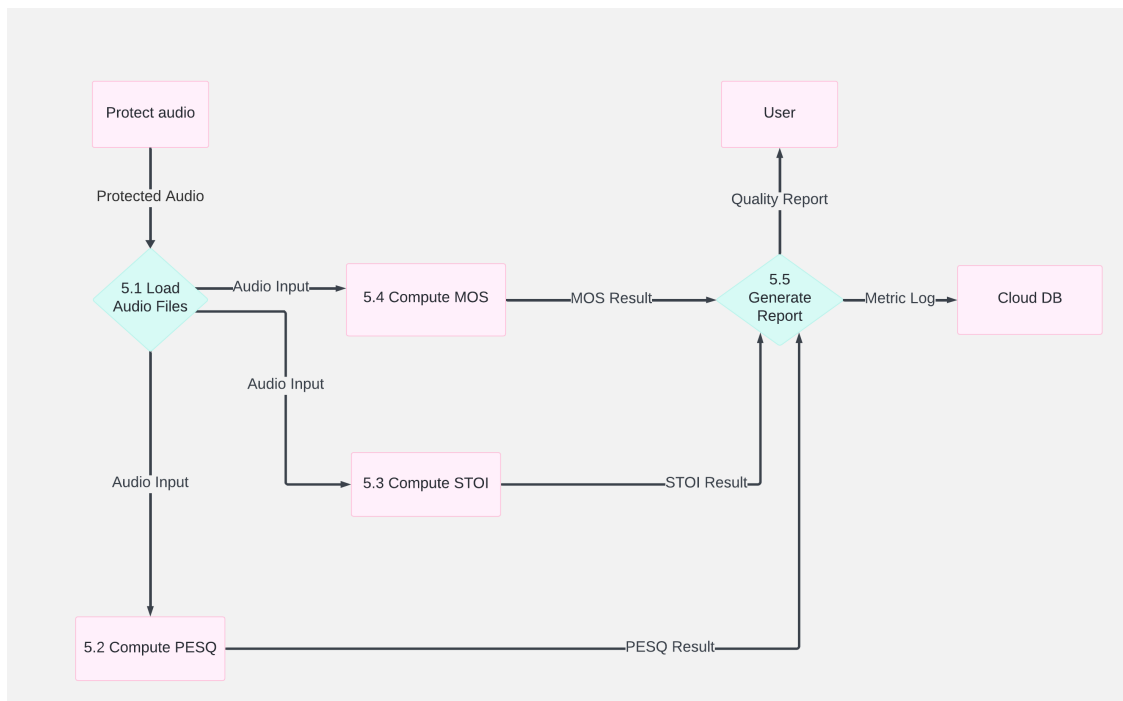


Figure 3.19: Level 2 DFD — Quality Evaluation Workflow

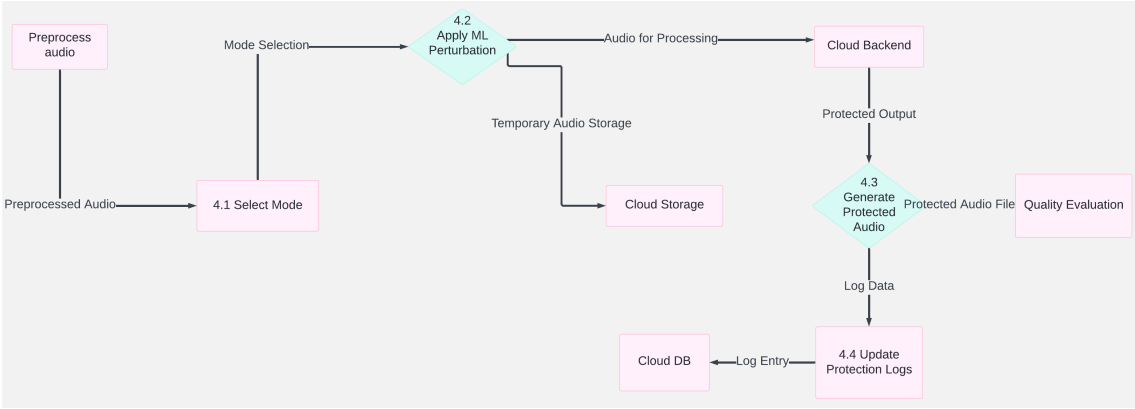


Figure 3.20: Level 2 DFD — Audio Protection Workflow

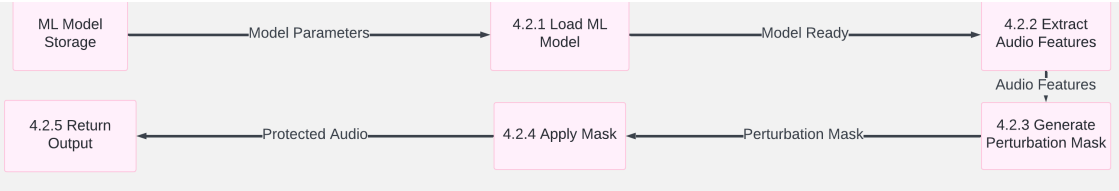


Figure 3.21: Level 3 DFD — ML Perturbation Process

Description: The Data Flow Diagrams provide a hierarchical view of how data traverses through the FakeLess system. Starting from user input and recording (Level 0-1), it decomposes into detailed sub-processes such as audio protection, ML perturbation, and metric evaluation (Level 2–3). This layered view highlights the procedural flow of data between modules, databases, and external entities.

3.2.3 System Sequence Diagrams

This section presents the System Sequence Diagrams (SSDs) for all 15 major use cases of the Fakeless application. Each SSD illustrates how the user, the mobile app, and the backend interact to achieve a particular goal.

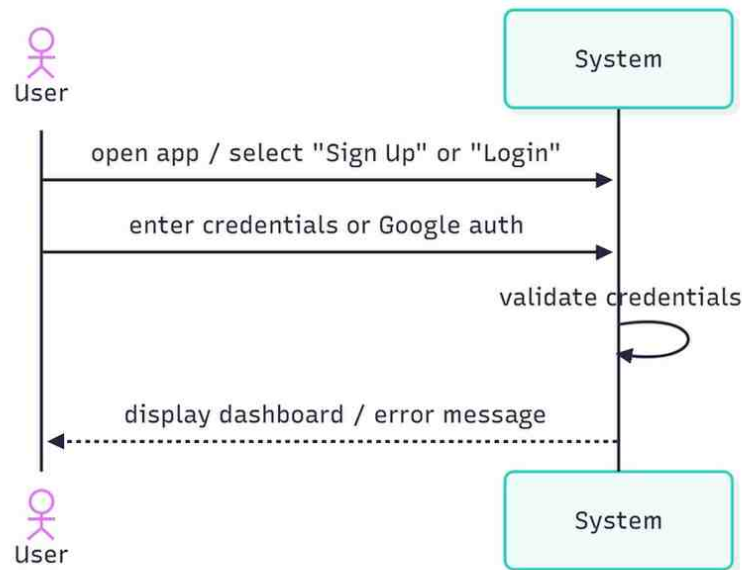


Figure 3.22: SSD for Use Case 1: User Registration and Login

Description: This sequence shows how a user creates an account or logs in through email or Google authentication. The system verifies credentials with the backend before granting access.

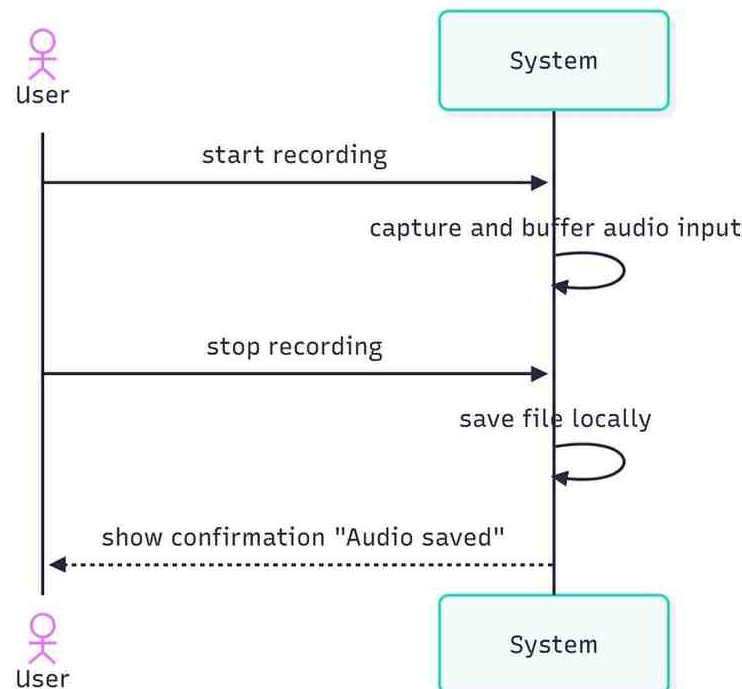


Figure 3.23: SSD for Use Case 2: Record Audio

Description: The user initiates the recording process. The app requests microphone permissions, records the audio, and saves it locally for further processing.

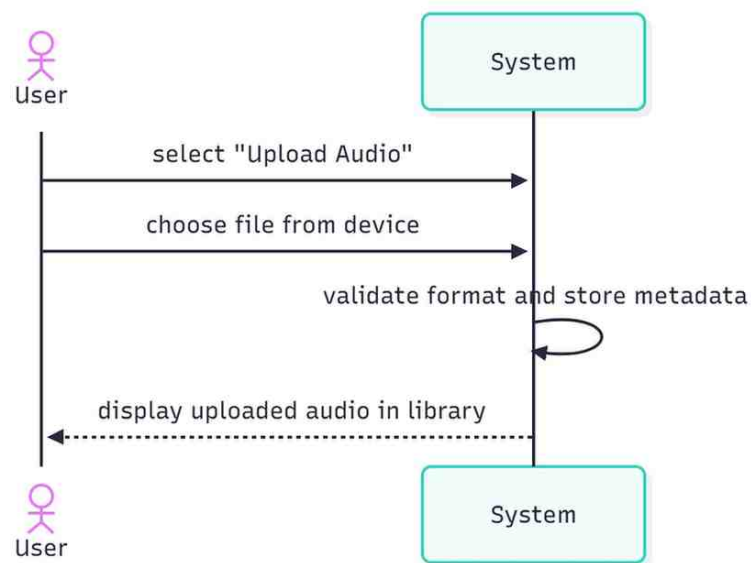


Figure 3.24: SSD for Use Case 3: Upload Existing Audio

Description: The user selects an existing voice file from storage. The app validates the format, uploads it, and stores metadata for later processing.

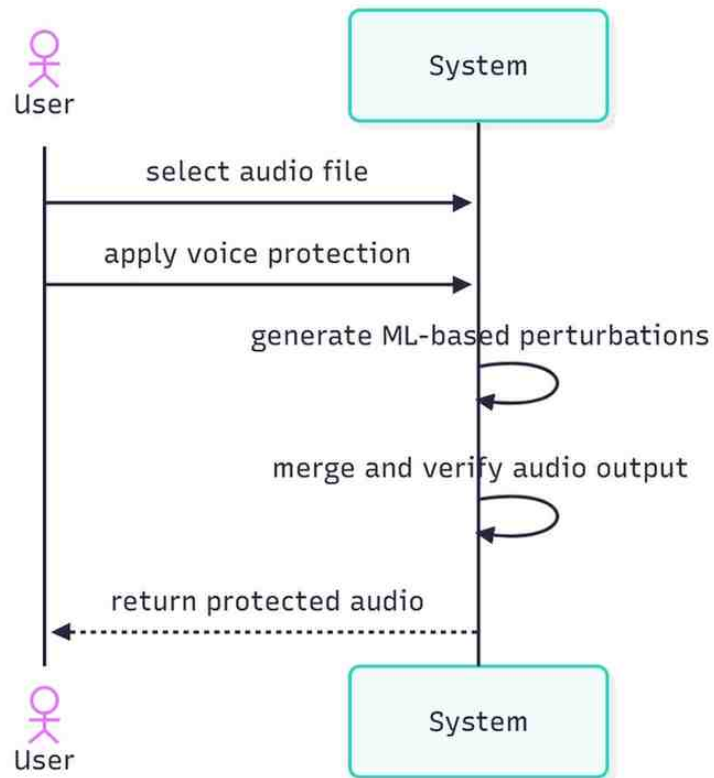


Figure 3.25: SSD for Use Case 4: Apply Voice Protection

Description: This SSD demonstrates how the app applies ML-generated perturbations to a selected audio file, sending it to the backend for model inference and receiving a protected version.

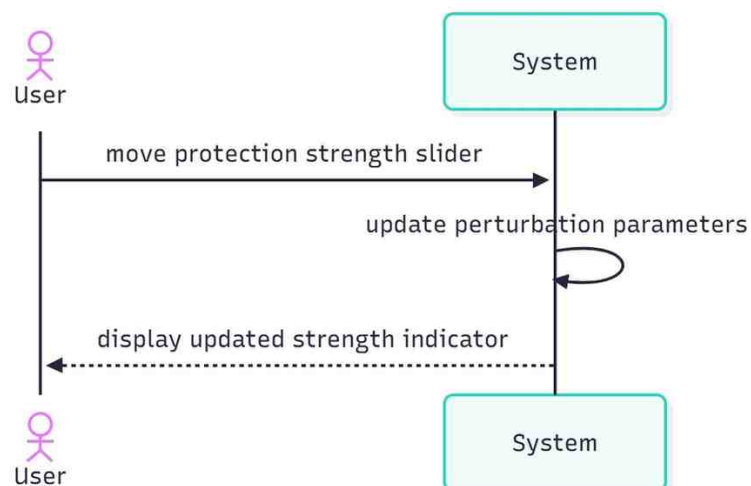


Figure 3.26: SSD for Use Case 5: Adjust Protection Strength

Description: The user adjusts the protection intensity via a slider. The system recalibrates the ML parameters and regenerates the protected audio accordingly.

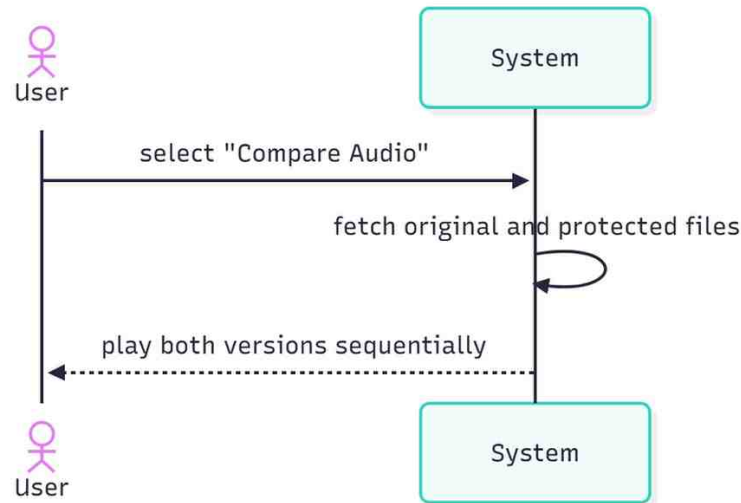


Figure 3.27: SSD for Use Case 6: Playback and Compare Audio

Description: The app retrieves both the original and protected audio, allowing the user to listen and compare them to verify quality and intelligibility.

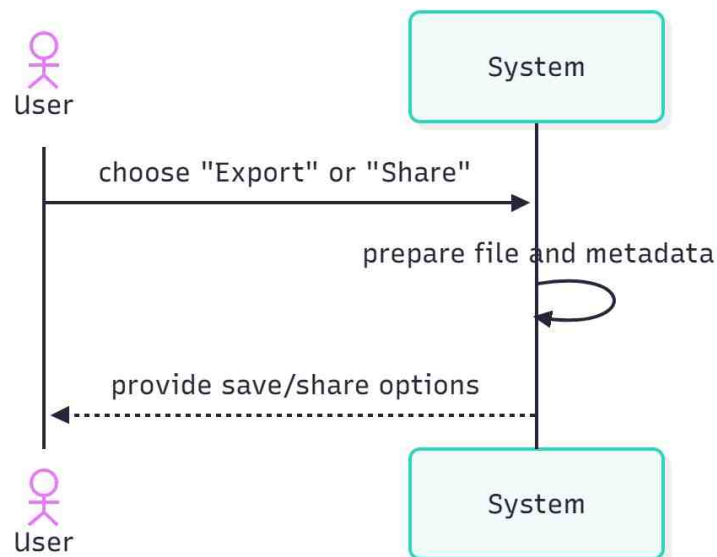


Figure 3.28: SSD for Use Case 7: Export or Share Protected Audio

Description: After protection, the user exports the final audio file to local storage or shares it via messaging and social apps using system intents.

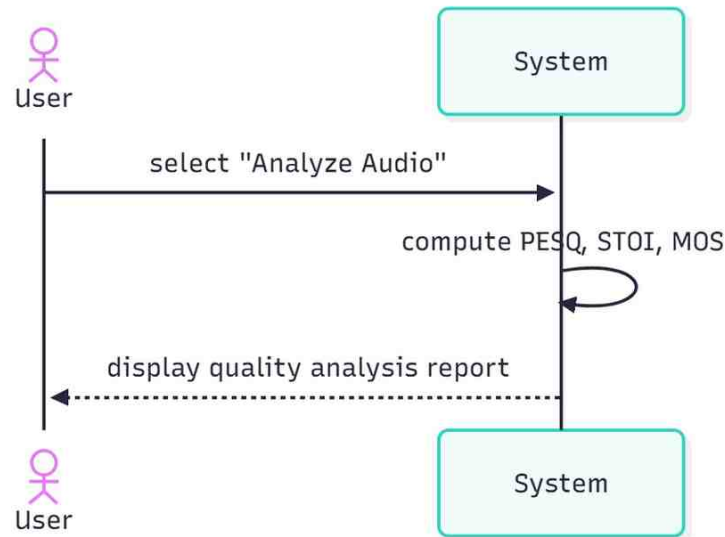


Figure 3.29: SSD for Use Case 8: Analyze Audio Quality Metrics

Description: The backend evaluates the protected audio using metrics like PESQ, STOI, and MOS, returning numerical scores and a summary report to the user.

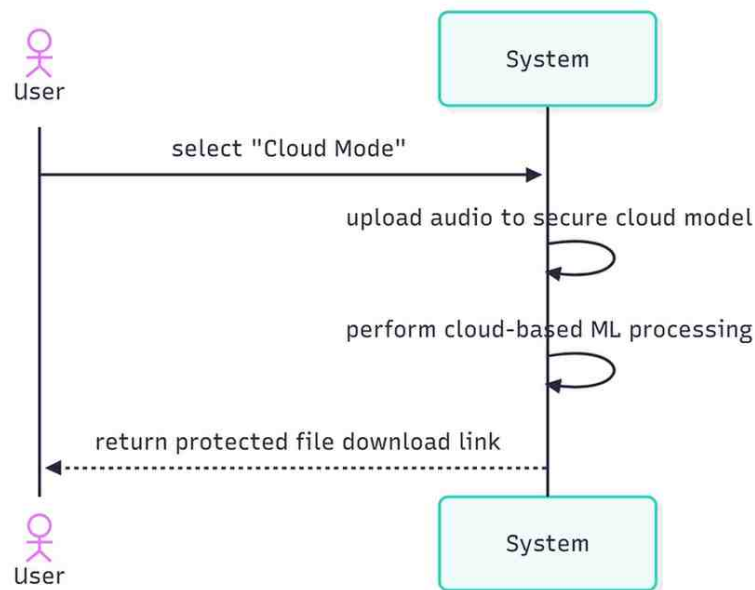


Figure 3.30: SSD for Use Case 9: Cloud Protection Mode

Description: When users opt for cloud mode, large audio files are uploaded to the backend for advanced ML protection. The processed file is later returned and stored securely.

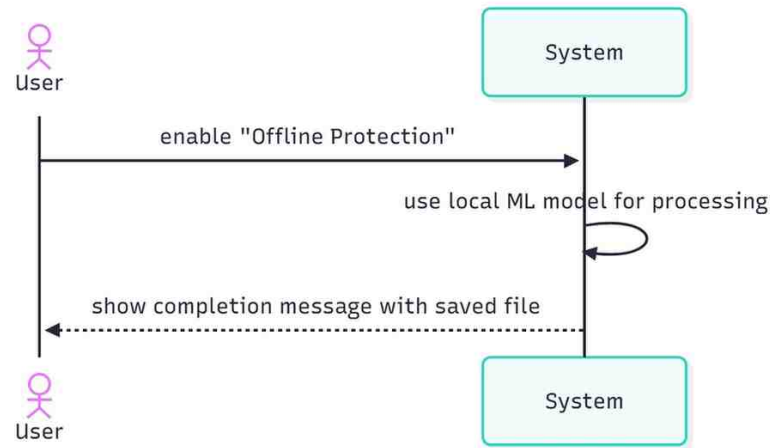


Figure 3.31: SSD for Use Case 10: Offline Local Protection

Description: For smaller audio files, the app performs protection locally on-device without needing an internet connection, preserving privacy and speed.

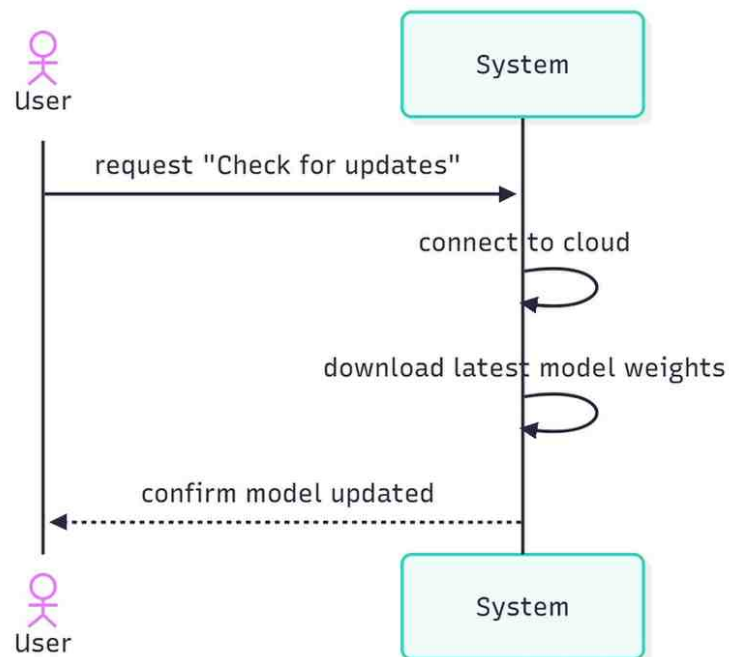


Figure 3.32: SSD for Use Case 11: Model Update Synchronization

Description: The app periodically syncs its ML models with the cloud to incorporate new anti-deepfake defense strategies, ensuring adaptability against evolving attacks.

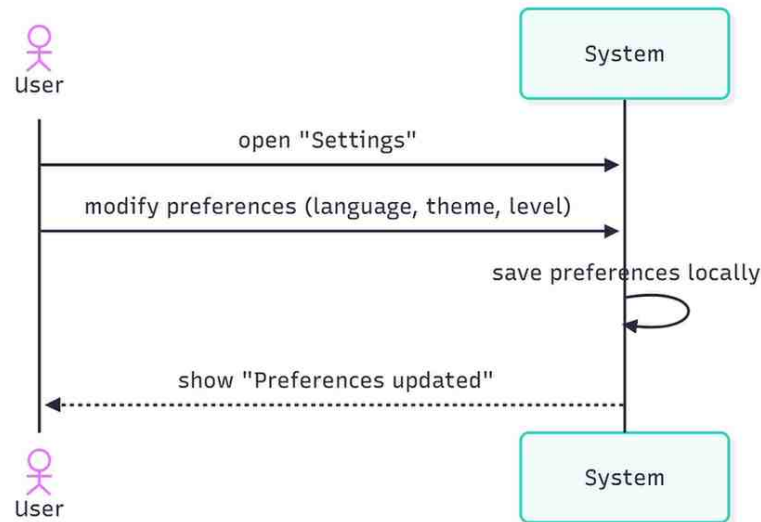


Figure 3.33: SSD for Use Case 12: Manage User Preferences

Description: Users can update app preferences such as language, theme, and default protection level. The system stores these securely and applies them across sessions.

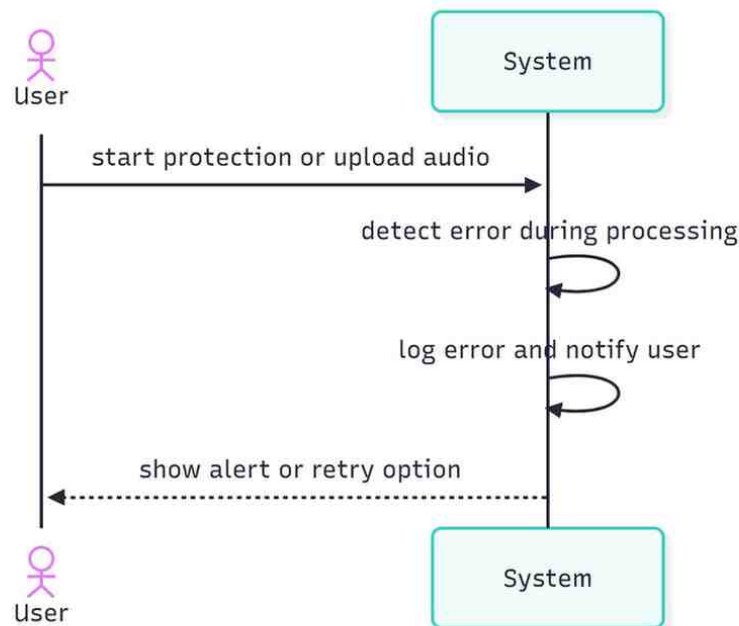


Figure 3.34: SSD for Use Case 13: Error Detection and Logging

Description: When an error occurs during processing, the system logs diagnostic information locally and, if permitted, sends anonymized reports to the backend for analysis.

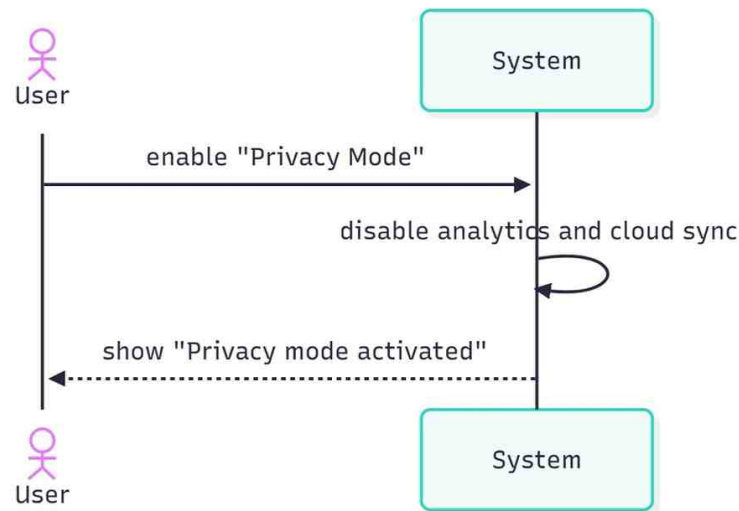


Figure 3.35: SSD for Use Case 14: Privacy Mode Activation

Description: This diagram illustrates the flow when a user activates Privacy Mode, which disables analytics and restricts all operations to local processing.

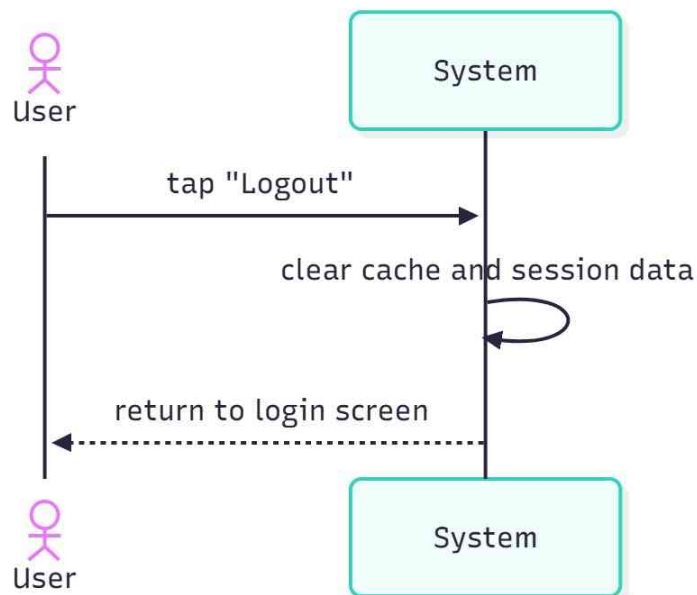


Figure 3.36: SSD for Use Case 15: Logout and Session Termination

Description: Upon logout, the system clears temporary data, terminates active sessions, and securely deletes cached audio to ensure user privacy.

3.2.4 State Transition Diagram

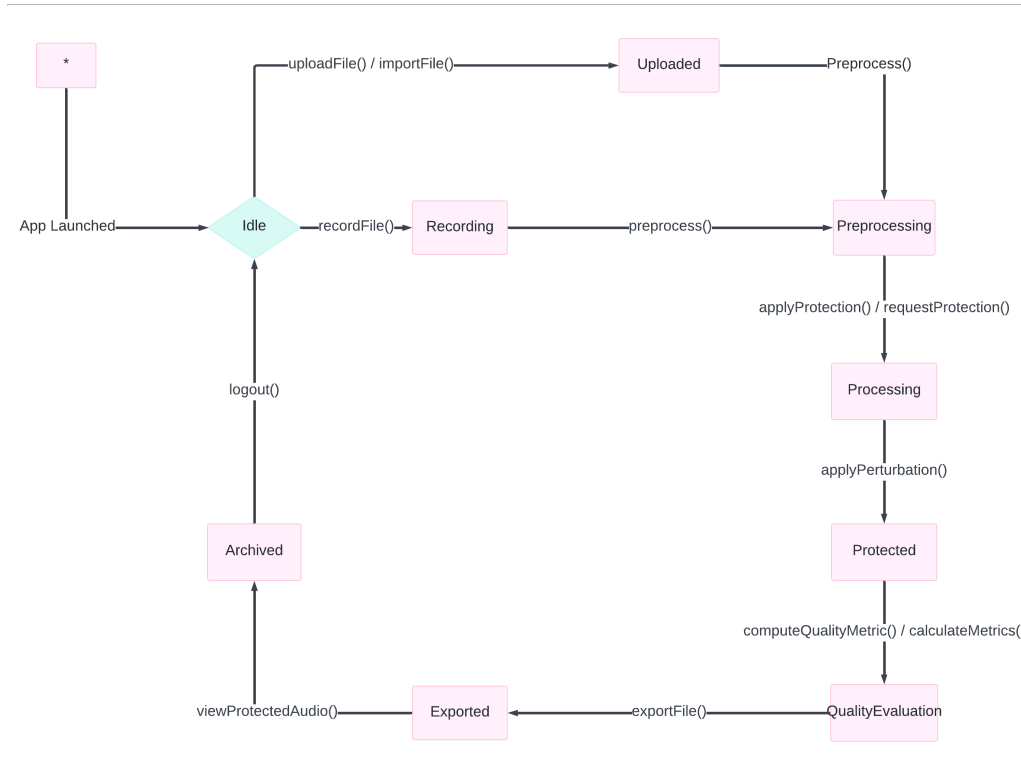


Figure 3.37: State Transition Diagram for Audio Processing States

States: *Idle* → *Recording* → *Processing* → *Protected* → *Exported* → *Archived*. Each transition occurs upon user action or system event such as protection completion or file export.

3.3 Data Design

The data design of FakeLess is centered around structured storage and efficient access of audio metadata, model outputs, and user preferences. The system employs both local and cloud storage solutions to ensure speed and security.

3.3.1 Data Storage Overview

- **Local Storage (SQLite):** Used for storing user settings, logs, and small audio files.
- **Cloud Storage (AWS S3 / Firebase):** Used for protected audio, model versions, and training datasets.

- **Backend Database (PostgreSQL):** Manages user profiles, metadata, and analytics securely.

3.3.2 Structures

- **User:** {user_id, name, email, password_hash, preferences, created_at}
- **AudioFile:** {file_id, user_id, filename, format, duration, status, created_at}
- **Perturbation:** {perturb_id, file_id, type, strength, applied_at}
- **MLModel:** {model_id, version, type, parameters, accuracy, last_updated}
- **QualityMetrics:** {metric_id, file_id, PESQ, STOI, MOS, evaluation_date}

3.3.3 Data Flow Description

1. User records or uploads an audio file, which is temporarily stored in local memory.
2. Audio metadata is inserted into the database with unique identifiers.
3. Audio is processed by the ML engine (local or cloud).
4. Perturbation details and quality metrics are logged in the database.
5. Final output and reports are available for playback, export, or archival.

3.3.4 Data Design Principles

- Minimal redundancy through normalization.
- Secure encryption for sensitive data.
- Use of ORM (Object Relational Mapping) for consistent backend operations.
- Clear separation between transient and persistent data.

Bibliography

- [1] Andrea Birrer and Natascha Just. What we know and don't know about deepfakes: An investigation into the state of the research and regulatory landscape. *New Media & Society*, 2024.
- [2] Adrián Gambín et al. Deepfake video detection: challenges and opportunities. *Artificial Intelligence Review*, 2024.
- [3] Saad Hussain et al. Understanding and mitigating audio adversarial examples. In *USENIX Security Symposium*, 2021.
- [4] Ye Jia et al. Real-time voice cloning. In *International Conference on Learning Representations (ICLR)*, 2019.
- [5] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations (ICLR)*, 2018.
- [6] McAfee LLC. Deepfake detection api for audio and visual content. Product documentation, 2023.
- [7] Reality Defender Inc. Multi-modal deepfake detection platform for enterprise security. Technical white paper, Reality Defender, 2022.
- [8] Sensity AI. Visual threat intelligence: Detecting deepfakes at scale. Industry report, Sensity AI Research, 2021.
- [9] Jonathan Shen et al. Natural tts synthesis by conditioning wavenet on mel spectrogram predictions. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4779–4783, 2018.
- [10] Aaron van den Oord et al. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016.
- [11] Jinchao Yi, Chen Wang, Jianhua Tao, X. Zhang, C. Y. Zhang, and Y. Zhao. Audio deepfake detection: A survey. *arXiv preprint arXiv:2308.14970*, 2023.

- [12] Jinchao Yi, Chen Wang, Jianhua Tao, X. Zhang, C. Y. Zhang, and Y. Zhao. Safeear: Content privacy-preserving audio deepfake detection. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, 2024.
- [13] Zhen Yu, Sheng Zhai, and Ning Zhang. Antifake: Using adversarial audio to prevent unauthorized speech synthesis. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 460–474, 2023.